

Portfolio SPA Architecture & CI/CD

Technical Documentation v1.0

1. Architecture Overview

This Single Page Application (SPA) is hosted using a serverless, highly available architecture on AWS. The core design principles are security through private origins and performance via edge caching.

- **Amazon CloudFront:** Serves as the global entry point, providing SSL termination and edge caching.
- **Amazon S3 (Private):** Acts as the origin server. Public access is disabled; content is reachable only via CloudFront.
- **Origin Access Control (OAC):** The mechanism that signs requests from CloudFront to S3, ensuring only the CDN can fetch objects.

```
graph LR
    User((User)) -- HTTPS --> CF[CloudFront CDN]
    subgraph "AWS Infrastructure"
        CF -- OAC Signed Request --> S3[(S3 Bucket - Private)]
        ACM[AWS Certificate Manager] --- CF
    end
    subgraph "CI/CD Pipeline"
        GH[GitHub] -- Webhook --> CP[CodePipeline]
        CP --> CB[CodeBuild]
        CB -- Upload Artifacts --> S3
        CB -- Invalidate Cache --> CF
    end
```

2. Workflow & CI/CD Strategy

The automated deployment pipeline ensures that any change pushed to the repository is tested, built, and synchronized to the production environment without manual intervention.

1. **Source Stage:** GitHub notifies AWS CodePipeline of a new commit via webhooks.
2. **Build Stage:** AWS CodeBuild provisions a managed container, installs dependencies (npm/yarn), runs the build script, and generates production assets.
3. **Deploy Stage:** CodeBuild syncs the `/dist` folder to the S3 bucket.
4. **Activation Stage:** A final build command triggers a CloudFront **Invalidation** to clear old cached files at the edge.

```
sequenceDiagram
    participant Dev as Developer
    participant GH as GitHub
    participant CP as CodePipeline
    participant CB as CodeBuild
    participant S3 as S3 Bucket
    participant CF as CloudFront

    Dev->>GH: git push origin main
    GH->>CP: Source Trigger
    CP->>CB: Start Build Job
    Note over CB: Install & Build
    CB->>S3: aws s3 sync ./dist s3://bucket
    CB->>CF: Create Invalidation (/*)
    CP->>Dev: Deployment Complete
```

3. Scaling Strategy

- **Elasticity:** CloudFront and S3 are inherently serverless and scale automatically from dozens to millions of requests without manual provisioning.
- **Performance:** Use of **Edge Locations** reduces latency by serving the SPA from the nearest geographical point to the user.
- **Build Concurrency:** CodeBuild can execute multiple builds in parallel if the portfolio expands to include multi-branch staging environments.

4. Security & Threats

Category	Implementation	Threat Mitigated
Access Control	OAC + S3 Block Public Access	Prevents direct S3 URL access; enforces CDN policies.
Data in Transit	ACM Managed SSL (TLS 1.2+)	Protects against packet sniffing and MITM attacks.
Infrastructure	IAM Roles (Least Privilege)	Limits blast radius if CI/CD credentials are leaked.
Frontend	Security Headers (HSTS, CSP)	Mitigates XSS, Clickjacking, and Protocol Downgrades.